

FinalProjectReport

Petko Petkov

April 2025

1 Introduction

Non-inclusive language in job descriptions has been a deterrent for many candidates. The use of gender-exclusive language has been found to greatly decrease the diversity of applicants [1]. With such negatives, it's surprising that this problem is so widespread; a study by Romano et al[2] found that a significant number of processed job descriptions required refinement. JobFair aims to eradicate this problem by using Alice Ashcroft's extensive research on gender inclusive language, in conjunction with an LLM, to produce a SaaS that helps recruiters craft and edit inclusive job descriptions.

1.1 Related Works

During my placement, I helped with the development of the current MVP workflow, which can be broken down into two parts:

1. Generating a job description through a Q&A approach
2. A user uploads an existing job description, which is analysed by our agentic system and produces a list of recommendations the user can apply

1.1.1 Workflow 1

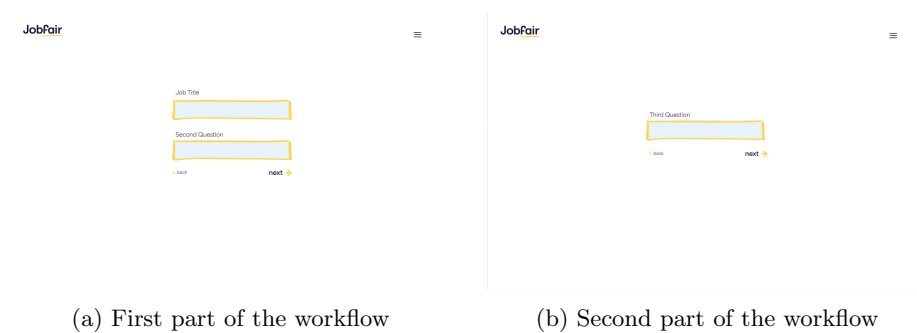


Figure 1: Complete two-step workflow

1 shows the pre generation part of the workflow, please note that these are design documents and so 1a has a title field and two question forms instead of one. The user begins by entering the job title for their desired description, followed by answering two preset questions:

1. What skills are essential for your ideal candidate to possess
2. What skills are nice to have

Once the user has answered the two preset questions an LLM uses the aggregated question answer pairs to generate a third question for the user to answer about their job description. Finally a job description is generated by using the aggregate of all three question answer pairs.

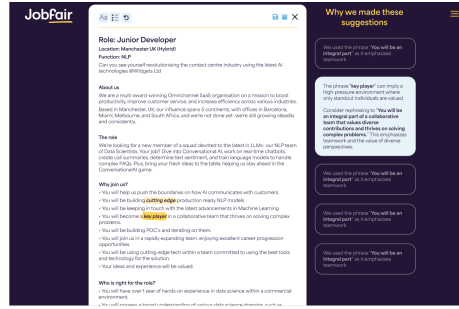


Figure 2: Post generation screen

Figure 2 shows the user interface post job description generation. A transparent side panel that exposes each suggestion’s rationale helps recruiters build an accurate mental model of how the system works, which in turn drives higher trust, lower cognitive load, greater satisfaction, and stronger compliance. By surfacing concise, contextual explanations immediately adjacent to generated copy, JobFair not only demystifies its inclusive-language engine but also transforms every interaction into a learning moment—enabling recruiters to internalize best practices over time. This level of explanation and transparency is what separates jobFair from other competitors who often adopt a black box approach. The explanations provided by job fair builds aims to build trust with the user [3]. In addition to this by providing both “why” and “how” explanations side-by-side we can significantly improve user satisfaction and perceived usefulness [4]

1.2 Problems with the current system

Despite achieving the goal of allowing users to create an inclusive job description the current MVP exhibits several problems that hinder the quality at which it achieves this goal:

1. **Overly rigid interaction model.** All users are funneled through the same two preset questions, which limits the system’s ability to capture role-specific nuance. This narrow context often results in formulaic, “one-size-fits-all” job descriptions that fail to reflect the true needs of the position.
2. **Incomplete requirement gathering.** Key sections of a well-crafted posting—such as compensation ranges, benefits, and core responsibilities—can be unintentionally skipped because the static Q&A flow doesn’t prompt recruiters to supply these details. As a result, the quality of the output becomes overly dependent on each user’s individual expertise, leading to inconsistent and potentially misleading job adverts.
3. **Absence of domain guidance.** Most recruiters don’t possess the depth of technical knowledge to understand nuances in requirements for the role they are hiring. The current system has no way of aiding the user throughout the requirements gathering stage
4. **Cumbersome revision process.** After a job description has been generated should the user wish to edit it, they must use a separate editing software, and then re import it into jobfair for further analysis. This repeatedly fractures the user’s workflow -fragmenting their “working-spheres”, driving up cognitive load and undermining productivity[5]

1.3 Aims and Objectives

The proposed solution aims to deliver a redesigned, full-stack SaaS workflow that eliminates the shortcomings identified in Section 1.2 by coupling an agent-based generation engine with a user-centred interface.

1. **Back-end architecture:** Design, implement and evaluate an agentic pipeline that gathers requirements, retrieves relevant exemplars, iteratively generates inclusive job descriptions, and scores them against predefined criteria.
2. **Front-end experience** – Create an intuitive interface that integrates conversational input, and allows post job description generation editing, eliminating the need for external editors.
3. **User-driven refinement** – Conduct iterative usability sessions with recruiters and hiring managers; use their feedback to prioritise features and fine-tune both engine and interface.

In the rest of this paper I will be focusing on rationale behind design choices and how the end system aims to improve upon the 4 identified pitfall of the previous MVP as stated in 1.2.

2 Methodology

We adopted a Design-Science Research (DSR) approach—building artefacts to solve identified problems—while aligning each build phase with the ISO 9241-210 human-centred design activities. DSR provides the research backbone (problem relevance, rigorous evaluation, transparent contribution) [6], and ISO 9241-210 ensures that even when formal user testing occurs only at the end, every feature is still grounded in recruiter needs [7]. Three internal build cycles addressed the four MVP shortcomings: Cycle 1 produced a slot-filling chatbot plus Retrieval-Augmented Generation (RAG) and an iterative judge to replace the rigid, under-specified flow; Cycle 2 wrapped that engine in a Streamlit proof-of-concept for rapid stakeholder feedback; Cycle 3 delivered a React canvas that integrates in-place editing, version history, explainable suggestion cards, and highlight-to-edit—all aimed at removing context-switch costs and boosting trust. Only the fully-integrated artefact underwent a summative SUS+NASA-TLX study, thereby satisfying the DSR evaluation guideline while meeting ISO’s user-validation requirement.

2.1 Agentic Back-end

The new agentic system fixed all 4 of the problems stated in 1.2. The rest of this section will breakdown the design rationales behind how each one was solved followed by an explanation of the technical implementation.

2.2 Rigid Q&A Flow - Design & Rationale

The first problem was improving the rigid workflow, figure 3 shows the agentic workflow of the MVP:

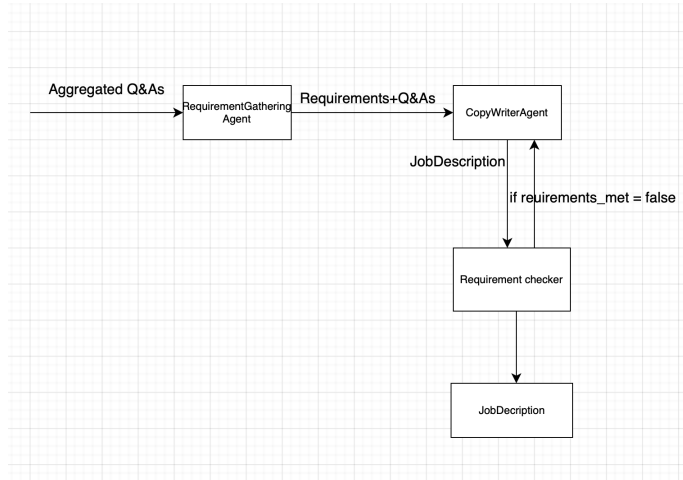


Figure 3: Agentic System for generating a job description

1. A requirements gathering agent extracts a list of requirements from the answered questions
2. The list of requirements in conjunction with the Q&A pairs is provided as context to the copywriter agent who's goal is to generate a description
3. the generated description is then checked by a requirements checker agent, if the description covers all requirements it is passed out of the system, if not the copy writer agent continues to rewrite the description until the requirements have been met

The only user provided context in the system is the brief set of answers to the three questions. This isn't enough context for an LLM to generate a good quality domain-specific response. As research has extensively shown LLM's quality of output largely depends on rich structured prompts, without which the LLM is at higher risk of hallucinating and producing a boiler plate response [8]. This is because LLM's will frequently default to outputting boiler-plate tokens in the absence of meaningful context [5]. In addition to this LLM's are highly sensitive to minor variations in prompts, as the user provided context is so small, slight changes to wording or spelling can produce extreme fluctuations in the quality of the LLMs output [9].

To eliminate the rigid, three-answer funnel, we introduced a slot-filling chatbot that collects a rich, structured prompt before generation. A slot is defined as a specific structured piece of information, for example a list of key requirements, or a concise summary of the company. Slot filling is a well-established strategy in dialogue systems for turning free-text exchanges into complete, structured records [10] and has recently been adopted in requirements-engineering tools such as LadderBot, which more than doubled the amount of concrete user input compared to the alternative information gathering method - static forms [11]. By having this pre defined structured output consisting of slots the job description generator agent is provided with much more rich task specific context, which is the strongest predictor of factuality and specificity in an LLM's output [12]. The Chat bot would engage with users in conversation until the slots were all filled guiding them along the way this guided method of slot filling consistently scores the highest in usability ratings compared to alternative methods [13]. Once all information has been gathered, the slots would be passed to the job description generation agent. The rationale behind providing only the slots to the job description generation agent instead of the open ended dialogue between a user and agent was based off the findings of the prompt-SET benchmark, which found that when rephrasing a prompt by as little as 10 tokens swings accuracy by up to 76 percentage points, whereas supplying the same content as key:value *slot tables* cuts that variance by more than 40pp [14],[15]. Therefore with this design we aim to remove the influence of prompt sensitivity on the LLM's final output.

Even with a complete slot table, a large-language model still risks reverting to generic phrasing if it has no concrete linguistic examples for the target domain [16]. Therefore, we decided to implement a lightweight RAG layer that

feeds the model ways of phrasing the acquired slot information from a corpus of 400 job descriptions. RAG has been shown to *raise specificity*—Lewis *et al.* reported a 12-point factual F gain on knowledge-intensive QA when retrieved passages were concatenated to the prompt [17]. Follow-up work by Shuster [18] found a 29% drop in hallucinations on the Wizard-of-Wikipedia benchmark [?]. Because we retrieve *type-matched* requirements for each slot, the model is handed domain-specific verbs and noun phrases “*implement CI/CD pipelines using GitHub Actions*” instead of the generic “good DevOps skills”—thereby eliminating the formulaic phrasing produced by the MVP. This dual-pronged method of using slots together with RAG allows the LLM to learn semantic representations of slots which are transferable across domains and robust to misaligned schemas [19]

2.3 Incomplete requirements Gathering - Design&Rationale

Unstructured job adverts make it hard to match candidates accurately [20]; NLP analyses of over 42 million unstructured postings report significantly lower retrieval precision when listings lack an explicit schema [21]. The current MVP’s Q&A workflow extracts only three user inputs - job title, must-have skills, and nice-to-have skills, plus a single AI-generated question, which, as discussed in the section 2.2, suffers from prompt sensitivity. This approach led to two problems:

1. There is a lack of mandatory fields that should be included as part of a “good” job advertisement:
 - (a) A title that indicates the seniority position of the job [22]
 - (b) Specific key responsibilities and tasks [23]
 - (c) Compensation and benefits section, which includes pay information, boosting transparency, which in turn reduces gender and minority pay gaps [24]
 - (d) Work Location and Flexibility [25]: A lot of companies have started offering flexible working patterns such as remote or hybrid. The current MVP doesn’t encourage recruiters to advertise this.

This project includes all of these as part of the mandatory “slots” discussed above.

2. The current MVP puts the burden on the user, leading to inconsistent output quality. For example, the quality of the third AI-generated question is completely reliant on how much information the user has provided; the system provides no guidance in encouraging the user on how much information they should provide or guidance on how to structure it, other than the question itself, which serves as a vague prompt. This sort of system, which lacks structured, guided interfaces, forces the user to make quality-affecting decisions, which in turn increase the risk of failure [26]. The chatbot interface for requirements gathering removes this burden by

having the agent ask for clarification or elaboration from the user whenever more detail is required.

A counterargument to the implementation of this project is simply adding more form fields to the MVP, which the user must answer, essentially mimicking the "slots" implemented in this project. However, the method of collecting user information using "forms" has been proven to score lower in usability and user satisfaction when compared to chatbots [27]. In addition to this, as mentioned previously, ChatBots for requirements gathering remove the "burden on the user" as the agent will continue to prompt the user for information it still needs.

Lastly, the current system is much easier to alter. For example, as new research comes out on what should constitute a job description, e.g adding a new slot. The only alterations that need to be made are within the structured output of the LLM agent, while with the MVP, the front-end interface would need to change. This leads us to the conclusion that the new chatbot implementation should be better at gathering requirements and providing the job description generator agent with more context, as well as being easier to extend.

2.4 Judge Pattern For Self Refinement - Design & Rationale

Once the requirements have been gathered together with contextual examples gathered from RAG, the job description generator agent produces a first draft, which is evaluated by a Judge Agent. The judge agent's goal is to grade the output of an LLM against a marking criterion and provide feedback allowing the original agent to iteratively improve its output. The main rationale for this inclusion is to remove the negatives of one-shot prompting. The rest of this section will look at the drawbacks of one-shot learning and how that motivated the design of a judge agent for the new system, concluding with a rationale for the marking criteria used by the agent.

Despite richer prompts from RAG and the implemented slots in section 2.2, one-shot generation still produces hallucinations or biased phrasing:

1. A paper from Jiang et al.[28] on that knowledge-augmented LM's still often "ignore the given knowledge and hallucinate the answer", especially if the retrieved context is irrelevant or the model fails to use it. Their work on a verification module shows that without an active "checking" agent, an LLM might produce incorrect outputs despite having the right facts on hand. This justifies adding a Judge agent to cross-check factuality rather than relying on retrieval alone.
2. Although structured slot prompts and RAG narrow the variance space, prompt-sensitivity cannot be eliminated. Small lexical changes in a user's wording can still make the slot extractor emit slightly different JSON, which in turn nudges the retriever toward a different exemplar set. Ensemble-prompting and chain-of-thought (CoT) reasoning reduce, yet never abolish this residual fragility by averaging multiple prompts or forcing the model

to externalise its reasoning [29]. Our Judge pattern delivers the same robustness in situ: it externalises the model’s latent reasoning into an explicit 5-factor rubric and then iteratively refines the draft until that rubric is satisfied, effectively giving the pipeline an automated Cot-style feedback loop.

Another, more practical, risk of low-fidelity one-shot output is editorial drag on the recruiter. LLM’s struggle with generating content and adhering to a required structure simultaneously [30]. This can lead to a draft that is overly verbose, biased, or missing mandatory sections, forcing the user to:

1. embark on a manual clean-up
2. restart generation

Both of which contradict our goal of a seamless in-context workflow set out in 1.3. Empirical HCI work on fragmented work spheres shows that every forced context-switch (here: from “review” to “rewrite”) elevates mental workload and depresses productivity [31]. Even worse, the reason recruiters will be using our product is that they themselves lack deep knowledge of creating an inclusive job description. As a result of this, a poorly generated draft with subtle gender biased flows could be completely missed by the recruiters. By having a judge, we are introducing a self-correction loop, whereby the agent can recursively improve its output based on structured critique. This method has been shown to significantly increase the quality of the LLM’s output both in terms of factual accuracy/coherence [32], and human preference [33].

The Judge computes a **10-point score** from five categories, with two hard gates (Inclusivity ≥ 1.5 , FRE ≥ 60). Table 1 details the sub-criteria, point splits, and the empirical evidence that motivated each weight.

Table 1: Sub-criteria, weights, and rationale for each rubric bucket.

Bucket (max pts)	Sub-criterion (pts)	Rationale for inclusion
Clarity & Structure (2.0)	Clear headers (0.5)	Improves scanning speed of web users 2013 [34]).
	Logical flow (0.5)	Reduces cognitive load in task sequences (GOV.UK Content Design [35]).
	Bullet lists ≤ 7 (0.5)	Uses Miller’s 7 ± 2 rule to avoid overloading working memory.
	Consistent formatting (0.5)	Predictable layout aids comprehension/accessibility (WCAG 2.2 [36]).
Technical Accuracy (2.0)	Role-skill match (0.7)	Ensures the generation agent doesn’t hallucinate unrealistic requirements; flagged by Indeed Hiring Lab 2022.
	Stack specificity (0.7)	Vague stacks raise hallucination risk; precise tokens cut error (Jiang et al. 2023 [28]).
	Certifications justified (0.6)	Over-spec drives self-selection bias and legal risk (CIPD 2021 survey).
Inclusivity (2.0) <i>gate</i>	Gender Language	This is one of the main issues JobFair tries to tackle, gendered language has been proven to reduce diversity and number of applicants.
	Readability:Fre ≥ 60 (0.8)	Texts with higher FRE scores were found to be easier to read and quicker to understand [37], with thresholds of above 60 being considered ideal for broad comprehension, therefore broadening pool of candidates
Completeness (2.0)	Mandatory sections present & detailed (2.0)	Ensures the description uses all slot fields.
Engagement (2.0)	Compelling “About Us” (0.7)	Strong mission narrative boosts applicant numbers[38]).
	Growth narrative (0.7)	Clear training/career-path information correlates with higher accept-rates (LinkedIn Talent Trends 2022).
	Authentic tone (0.6)	Avoiding jargon increases perceived warmth/trust (HR Future 2021 buzzword survey; Lagowska 2023 [39]).

Point allocation. Each bucket totals 2 pts to give equal macro-weighting, while sub-weights mirror the empirical severity of their failure modes (e.g. 1.2 pts for biased language because it severely decrease applicant diversity [1]). The 7/10 pass bar we set was deduced by trial and error as we tried to balance job description quality and speed. From internal testing we found that at a threshold of 7/10 the job descriptions produced were at an acceptable quality. Thresholds higher than this required too many iterations of write,judge,refine cycle 2.6.3.

Hard-gate justification. Biased or unreadable copy undermines *JobFair*’s DEI mission, hence Inclusivity and Readability are non-negotiable. A FRE 60 score maps to an 8th-grade reading level—recommended by both UK Gov Plain-English guidance and US CDC health-literacy standards.

Extensibility. Because the rubric is encoded as part of the system prompt for the agent, we could easily change the criteria to fit with new emerging research, the derived rubric is a good start based off empirical evidence.

This concludes the pre-generation part of the magnetic system, in the net section we will discuss design choices for the post-generation phase/editing, after which we will discuss the implementation of the system

2.5 Cumbersome Editing - Design&Rationale

Limitations in the MVP Editing is not a peripheral convenience but a core phase of every creative workflow: writers revise early drafts to refine structure and voice, designers iterate prototypes to converge on user needs, and engineers refactor code to improve quality. Multiple studies show that meaningful creativity emerges only through repeated cycles of draft-and-revision, each cycle allowing practitioners to externalise ideas, receive feedback, and progressively improve the quality of their work [40].In practice, modern creative teams rely on dedicated version-control infrastructure precisely because it captures fine-grained change history, enables safe experimentation, and supports accountability during collaborative editing [5]. By forcing recruiters to export a draft, make edits elsewhere, and then re-upload the file to our system, the MVP breaks this iterative improvement loop. Consequently, the very functionality recruiters need most remains outside the system’s boundary.

Design Objectives From section 2.5 we can deduce the following design goals for the editing part of the user workflow:

MVP Limitation (from 2.5)	Design Goal
No in-app revision loop	Enable in-context editing without leaving the SaaS UI
No change provenance	Provide persistent, navigable version history
Risky experimentation	Offer safe “branch-and-revert” workflow to encourage exploration

Table 2: A table of old MVP editing flaws

The figure shows the design of how the agentic system supports all of these design goals. The system can be broken down into two parts: **Editing Agent**. The editing agent provides recruiters with an *in-context, conversational* workspace that removes the round-trip to an external word-processor and therefore closes the revision loop identified in Table 2. Conversational UIs lower extraneous cognitive load by allowing users to express intent in natural language instead of manipulating complex toolbars or modal dialogues [41]. Crucially, the agent *maintains conversation history* across turns, similarly to the design of the Chatbot Agent discussed in section 2.2. Context-aware dialogue models that cache prior messages generate more coherent, on-topic responses and require users to re-enter less information than single-turn systems [42]. By providing the Agent with conversational context of the job descriptions it has produced, we allow it to use previous versions based on the user’s request, e.g, the user says to use the introduction of version two but the style of writing in version 3. The agent will be able to combine these versions, providing the user with more control over making creative changes to job descriptions mirroring the “palette-of-versions” interaction documented in Upwelling’s writer-centric VCS [43] and in Serman et al.’s study of creative practitioners who reuse earlier iterations to fuel new ideas [44]. Although a linear, save-as-you-go version log might appear sufficient, Chan and Schunn’s large-scale study of conceptual genealogies shows that creativity gains emerge not from *single* “far” recombinations but from *second-order* iteration—i.e. building on ideas that themselves already integrate distant concepts [45]. In their data, first-generation far combinations had, on average, no positive effect on overall concept quality, whereas later generations that *iterated on those far combinations* produced reliably higher-quality, more novel solutions. Our version-history layer operationalises exactly that mechanism: every commit is addressable, and the editing agent can weave fragments from earlier, divergent drafts into a new composite (“use the introduction from v2 and the inclusive tone from v3”). By turning old commits into reusable building blocks rather than static snapshots, the system enables the indirect, multi-step recombination pathway that Chan & Schunn identify as critical for creativity, thereby aligning our design with empirical evidence while still satisfying the safe-exploration goal in Table 2.

2.6 Implementation

The following section will focus on the concrete implementation of the backend system, based on the design choices stated above. The backend was constructed entirely in Python. Although Python is less efficient than compiled languages like C++, or RUST for backend development, particularly in terms of execution speed[46, 47, 48], it is the language with the best support for constructing LLM-based applications [49]. The framework used for agent orchestration was Pydanticai, which is a lightweight dependency injection-based framework allowing agents access to tools and, most importantly, output validators. Although using the LLM’s API would have offered faster response times, this framework is model agnostic, allowing us to easily swap out models during testing, and

with LLM being such a quickly evolving field, the ability to swap out to the newest model with ease was a huge consideration point.

2.6.1 Slot Filling Schema & ChatBot agent

The Chatbot Agent is the biggest change from the old MVP product; its functionality can be broken down into three components

1. Structured Output for Slots:

```
from pydantic import BaseModel, Field
from typing import List

class JobRequirements(BaseModel):
    job_title: str
    seniority_level: str = Field(description="Seniority level for the specified role")
    responsibilities: str = Field(
        description="The responsibilities of the job, i.e. day in the life of the role"
    )
    job_requirements: Requirements
    job_salary: str = Field(description="The salary range for the job")
    job_location: str = Field(description="The location of the job")
    job_type: str = Field(
        description="The type of job (e.g. full-time, part-time, contract, etc.)"
    )

class Requirements(BaseModel):
    essential_skills: List[SkillRequirement]
    soft_skills: List[SoftSkillRequirement]
    nice_to_have: List[NiceToHaveRequirement]

class SkillRequirement(BaseModel):
    type: str = "skill_requirement"
    requirement: str = Field(description="The skill that is an essential skill")

class SoftSkillRequirement(BaseModel):
    type: str = "soft_skill_requirement"
    requirement: str = Field(description="The skill that is a soft skill")

class NiceToHaveRequirement(BaseModel):
    type: str = "nice_to_have_requirement"
    requirement: str = Field(description="An optional nice-to-have skill")
```

The Chatbot agent uses Pydantic models to provide structured output, more specifically, its output is of type JobRequirements. The Pydantic library allows for complex relationships between classes and provides the agent with context as to what's required in each section e.g job salary has a description telling the agent to input "The salary range for the job". By using this method we ensure consistency in the agent's final output, a problem which has plagued LLM since its conception [50].

2. **System Prompt:** Pydantic enables the definition of a system prompt alongside a user prompt. The system prompt, configured by the developer, serves as primary instructions that strongly influence the LLM's behavior, thereby reducing susceptibility to prompt injection attacks [51].

```
CHAT_AGENT_PROMPT = """You are an expert HR chatbot focused on
    gathering the following information from the user:
YOU NEED TO GATHER THE FOLLOWING INFORMATION:
- Role title and level
- Essential technical skills (max 7)
- Essential soft skills (max 5)
- Nice-to-have skills
- Responsibilities
- Job Salary
- Job Location
- Job Type
CRITICAL:
-IF you don't have all the necessary information, return a string
    as your response, this response should be a message to the user
    asking for more information.
-IF you have all the necessary information, return a
    JobRequirements object.
-Ask one question at a time from the user.
-Keep track of what information you have and what you still need
    using the message history context.
-Use inclusive language in your questions.
"""
```

Above is the carefully crafted system prompt, we utilised best prompt engineering practices by providing context to the agent at the start of the prompt [12]. The prompt is well structured with proper spacing, aiding the model with correct tokenization and processing of the prompt. Lastly we've ensured the prompt is clear and instructional, with two pathways the model can take based on whether the slots have been filled, i.e if all slots are filled return a JobRequirements object, if not provide useful feedback to the user. Having a clear, structured system prompt is of paramount importance for the quality of the LLM's output. Especially for solving complex multi step reasoning tasks [52].

3. **Memory and Conversation History:** Pydantic agent calls are single instance calls to the specified LLM's API, meaning by default the agent has

no capability of remembering context from previous calls. The framework has an extremely easy way of implementing this; after running an agent, you can access the messages exchanged during that run from the *result* object [53].

However, the *result* object encompasses more than just user and assistant messages. It also includes system-level components such as *SystemPromptPart* and tool call metadata, which are primarily intended for internal orchestration and guiding the LLM's behavior [?]. While these elements are crucial for the agent's operation, they are not directly relevant to the conversational context we aim to maintain. Incorporating them into the conversation history can introduce noise, potentially leading to less coherent or contextually appropriate responses from the LLM.

```
new_message = Message(
    user_input=query,
    assistant_response=response
)

# Add message to appropriate history based on type
if history_type == "chat":
    thread.chat_history.append(new_message)
    # Get new messages from the result and filter out
    # SystemPromptPart
    new_model_messages = filter_system_prompts(result.new_messages())
    # Add filtered new messages to thread's chat_model_history
    thread.chat_model_history.extend(new_model_messages)

def filter_system_prompts(messages: List[ModelMessage]) -> List[
    ModelMessage]:
    """
    Filters out SystemPromptPart from ModelRequest objects

    Args:
        messages: List of ModelMessage objects

    Returns:
        List of ModelMessage objects without SystemPromptPart
    """
    filtered_messages = []

    for msg in messages:
        if isinstance(msg, ModelRequest):
            # Filter out SystemPromptPart
            filtered_parts = [part for part in msg.parts if not
                             isinstance(part, SystemPromptPart)]
            if filtered_parts: # Only add if there are parts left
                filtered_messages.append(ModelRequest(parts=
```

```

        filtered_parts))
    else:
        filtered_messages.append(msg)

    return filtered_messages

```

To address this, we implemented a filtering mechanism that processes the messages retrieved from the `result` object keeping only `ModelRequest` objects which contain the user query and agent response. This way we can maintain the pertinent user and assistant exchanges and provide them to the agent as context, allowing it to be aware of which information it still requires, and avoids asking the user to enter redundant, duplicate information.

2.6.2 Retrieval Augmented Generation

As mentioned in section 2.2 we implemented Retrieval Augmented Generation in the agentic system, providing the job description generation agent with additional context. The implementation of the Retrieval-Augmented Generation (RAG) mechanism within the job description generation system employs a sophisticated vector database architecture that leverages semantic similarity to augment user-provided job requirements with contextually relevant skills. This system operates through the `analyze_requirements` function, which serves as the core component of the RAG pipeline.

The function accepts a `JobRequirements` object - a structured representation of job-related data containing essential, soft, and nice-to-have skill requirements along with position metadata such as job title, seniority level, responsibilities, salary range, location, and employment type. Within the function, a three-stage RAG process unfolds:

1.Vectorization Stage The system systematically processes each requirement by generating dense vector embeddings using OpenAI's `text-embedding-3-small` model. This model maps each textual skill requirement to a high-dimensional vector space (1536 dimensions) where semantic relationships between skills are preserved as geometric relationships. The embedding generation occurs through an API call to OpenAI's embedding service:

```

embedding_response = deps.openai_client.embeddings.create(
    model="text-embedding-3-small",
    input=skill.requirement,
)
skill_embedding = embedding_response.data[0].embedding

```

2.Retrieval Stage For each vectorized requirement, the system queries a Supabase vector database using a Remote Procedure Call (RPC) to the `match_requirements` function. The database has the `pgvector` extension (version 0.8.0) installed, enabling efficient similarity searches over high-dimensional vectors. The query includes five critical parameters:

```
similar_skills = deps.supabase.rpc(
    'match_requirements',
    {
        'query_embedding': skill_embedding,
        'match_count': 2,
        'job_title_filter': requirements.job_title,
        'requirement_type': skill.type
    }
).execute()
```

The RPC interface accepts the query embedding vector, a limit on number of matches to return (2), a job title filter for contextual relevance, and the requirement type (to ensure like-for-like matching). Internally, the Supabase function performs a cosine similarity search against the `vector_table`, which stores job requirements as JSON data and their corresponding vector embeddings. Each entry in this table contains:

- `job_title`: Text field identifying the position
- `requirement`: JSONB field containing the requirement type and text
- `requirement_embedding`: Vector field storing the pre-computed embedding

3. Augmentation Stage The retrieved similar skills are processed and categorized by their type (essential, soft, or nice-to-have). For each retrieved skill, the system extracts the skill data and its similarity score:

```
for result in similar_skills.data:
    data = result['requirement']
    similarity = result['similarity']
    print(f"Similarity: {similarity}")
    if data['type'] == 'skill_requirement':
        final_essential_skills.append({
            'type': data['type'],
            'requirement': data['requirement']
        })
    # Similar code for soft_skill_requirement and
    # nice_to_have_requirement
```

The processed skills are then assembled into an `UpdatedRequirements` object that maintains the original structure but with augmented content. This object is used in the subsequent job description generation phase, where both the original user-provided requirements and the RAG-enhanced requirements are presented to the LLM:

```
generation_prompt = JOB_DESCRIPTION_GENERATION_PROMPT.format(
    job_title=requirements.job_title,
    original_essential_skills=chr(10).join(f'- {skill.requirement}' for
        skill in requirements.job_requirements.essential_skills),
```



```

    inclusive_essential_skills=chr(10).join(f'- {skill.requirement}' for
        skill in updated_requirements.essential_skills),
    # Similar formatting for other skill types
    # ...
)

```

The system utilises this dual representation approach to provide the LLM with both the user's original intent and contextually relevant skills that may have been overlooked. This implementation exemplifies a hybrid approach to content generation, where the LLM's generative capabilities are supplemented by a domain-specific vector database, ensuring that the resulting job descriptions contain professionally relevant and comprehensive skill requirements.

The database architecture supports this RAG pipeline through a specialised table schema that accommodates both textual and vector data, with appropriate indexing to facilitate efficient similarity searches. The implementation demonstrates how traditional relational database capabilities can be extended with vector operations to enable semantically aware content retrieval and generation.

2.6.3 Self-Refining Judge Model

The Judge Agent represents a critical component in our job description generation system. It enforces quality standards across multiple dimensions, ensures inclusivity, and drives iterative improvement of generated content. This section details our implementation, design decisions, and technical integration.

Master Prompt The Judge Agent relies on a detailed five-bucket rubric (rationale behind bucket choices in table 1) which is part of its system prompt.

```

JUDGE_AGENT_PROMPT = """
You are an expert job description evaluator. Your task is to grade job
descriptions
on multiple criteria. Please use the following detailed rubric to award
points...

1. Clarity & Structure (Max: 2 points)
  - Clear organization (0.5 point): Must have headers such as 'About Us
    ', 'Role'...
  - Logical flow (0.5 point): Content should progress naturally...
  - Mobile-friendly formatting (0.5 point): Use bullet lists with 7
    items per list.
  - Consistency of formatting (0.5 point): All sections are uniformly
    formatted.

2. Technical Accuracy (Max: 2 points)
  - Role-specific skills (0.7 point): Skills and technical requirements
    match...
  - Technology stack specificity (0.7 point): Specific tools and
    frameworks...

```

```

- Certifications justified (0.6 point): Only include certifications
  that are...

3. Inclusivity (Max: 2 points)
- Language use (1.2): Pick out 1-10 items of language this job
  description...
- Structural inclusivity (0.8 point): Flesch-Kincaid grade between 8
  and 10...

4. Completeness (Max: 2 points)
- All key sections present: Includes sections for DEI, growth
  opportunities...

5. Engagement (Max: 2 points)
- Compelling 'About Us' section (0.7 point).
- Growth narrative (0.7 point): Clearly describes opportunities like
  training...
- Authentic tone (0.6 point): Avoid corporate jargon...

CRITICAL:
-THE INCLUSIVITY MUST BE AT LEAST A 1.5 FOR A PASS
-THE FLESH READING EASE MUST BE AT LEAST A 60 FOR A PASS
"""

```

The prompt provides extensive examples for detecting non-inclusive language patterns, giving the agent both pattern recognition capabilities and explicit remediation strategies:

```

"""
Issue: Using "essential".
Solution: Think about what is actually essential, and what can be taught
...

Issue: Time taken to apply and number of stages in the process.
Solution: Expose the number of steps and think about why this number is
important...

Issues: Years of experience needed.
Solution: Think about the actually skills somebody needs...

Issue: "Flexible working"
Solution: What does this mean? For some people it's true flexibility...

Issue: "Enhanced parental/maternity leave"
Solution: What does this mean? Expose your policy on this...
Refer to parental leave as opposed to maternity leave or paternity leave.

Issue: Listing statutory holiday days as a "benefit"
Solution: Increase the amount of leave allowed by your employees...

```

```

Issue example: "5 years experience of SQL"
Solution: Do we need to list the programming language?...

Issue example: "experience working with teams"
Solution: This is vague and doesn't mean anything...
"""

```

This method of providing custom demonstrations to match the problem at hand ("tailor prompting") consistently outperforms generic examples, especially in complex tasks such as diagnosing language as gender biased [54],[55]. It is of crucial importance for these examples to be of high quality, as poor quality ones can actually decrease the performance of an LLM [56],[57]. To ensure our examples were relevant and high quality, we used Dr Alice Ashcroft's expertise to craft these examples, ensuring we are providing the high-quality examples the LLM needs.

Tool and Framework Orchestration The Judge Agent is implemented using a modern, modular architecture that leverages several specialized frameworks:

- **Pydantic-AI:** Provides the agent definition, type safety, and structured I/O
- **FastAPI:** Handles API endpoints and request validation
- **OpenAI API:** Powers the large language model (GPT-4o) for evaluation
- **textstat:** Performs readability analysis on job descriptions

The refinement flow follows these steps:

1. The job description agent generates an initial draft based on requirements
2. The draft is passed to the judge agent via the `evaluate_job_description` tool
3. The judge agent analyzes the text using its rubric and readability tool
4. A structured `JudgeResult` is returned with score and improvement items
5. If the score is below threshold (7/10) or fails critical criteria:
 - (a) The job description agent incorporates all feedback
 - (b) A new draft is generated with improvements
 - (c) The evaluation-refinement cycle repeats until passing criteria are met
6. The final approved job description is returned to the user

Structured Output The Judge Agent's output is formalized using Pydantic models, ensuring type safety and validation:

```
class FeedbackItem(BaseModel):
    category: str = Field(description="The evaluation criterion or category")
    item: str = Field(description="Specific area that needs improvement")
    reasoning: str = Field(description="Detailed reasoning behind flagging this area")
    recommendation: str = Field(description="Actionable recommendation for improvement")

class JudgeResult(BaseModel):
    score: float = Field(ge=0, le=10, description="Score out of 10")
    passed: bool = Field(description="Whether the job description meets requirements")
    feedback: str = Field(description="General summary feedback for improvement")
    improvement_items: List[FeedbackItem] = Field(
        default_factory=list,
        description="List of specific improvement items with detailed reasoning"
    )
)
```

Here's an example of the structured output:

```
{
  "score": 6.5,
  "passed": false,
  "feedback": "The job description needs improvements in inclusivity and readability.",
  "improvement_items": [
    {
      "category": "Inclusivity",
      "item": "Gendered language",
      "reasoning": "The phrase 'he must have' assumes male candidates.",
      "recommendation": "Replace with gender-neutral 'you must have' or 'the candidate must have'"
    },
    {
      "category": "Readability",
      "item": "Flesch Reading Ease",
      "reasoning": "Current score of 52.3 is below the minimum threshold of 60.",
      "recommendation": "Simplify sentence structure and use shorter paragraphs."
    }
  ]
}
```

Iterative Refinement Loop The judge is integrated into the job description generation workflow through a dedicated function exposed as a tool:

```
@job_description_agent.tool
async def evaluate_job_description(ctx: RunContext[Deps], description:
    str) -> JudgeResult:
    """
    Evaluate the quality of a job description using expert judge.
    Returns detailed score and feedback.
    Args:
        description: The complete job description you have produced as a
            string
    """

    evaluation_prompt = f"""
    Please evaluate this job description:
    Description: {description}
    """

    result = await judge_agent.run(evaluation_prompt, deps=ctx.deps)
    return result.data
```

The judge agent itself is constructed with precise type definitions, ensuring its outputs can be reliably consumed:

```
judge_agent = Agent[Deps, JudgeResult](
    'openai:gpt-4o',
    deps_type=Deps,
    result_type=JudgeResult,
    system_prompt=JUDGE_AGENT_PROMPT
)
```

For readability assessment, we implemented a specialized tool that calculates the Flesch Reading Ease score:

```
@judge_agent.tool_plain
async def evaluate_readability(description: str) -> str:
    """
    Evaluates the Flesch reading ease score of the job description.
    Args:
        description: The complete job description as a string
    """

    score = textstat.flesch_reading_ease(description)
    return f"Flesch Reading Ease Score: {score:.1f}"
```

Engineering Decisions Several key engineering decisions shaped the Judge Agent implementation:

- **LLM Selection:** We chose GPT-4o for its superior contextual understanding and ability to handle multi-dimensional evaluation tasks with nuanced scoring.

- **Hybrid Analysis:** Instead of relying solely on subjective LLM evaluation, we integrated objective metrics through the `textstat` library, creating a mixed-method approach that balances qualitative and quantitative assessment.
- **Structured FeedbackItems:** Rather than providing free-form feedback, we structured the output with specific categories, items, reasoning, and actionable recommendations—facilitating targeted improvements.
- **Pass/Fail Criteria:** We implemented explicit thresholds (7/10 overall, minimum 1.5/2 for inclusivity, 60+ for Flesch readability) to enforce minimum quality standards.
- **Memory-Augmented Refinement:** By instructing the job description agent to incorporate feedback cumulatively through the `message_history` mechanism, we prevent regression and ensure continuous improvement.

Limitations and Future Work Despite our robust implementation, several limitations remain. The Judge Agent’s evaluation is inherently subjective and may exhibit inconsistency in scoring across similar inputs. The rubric itself may drift from business requirements over time without regular calibration. Additionally, the iterative refinement loop lacks a hard cap on iterations, potentially leading to excessive API calls or diminishing returns on minor improvements.

A promising enhancement would be integrating a skills-ontology plugin that validates technical skills against industry databases. This would ensure skill requirements are current, relevant, and properly categorized by domain and seniority level. Such a plugin could catch deprecated technologies or inconsistent skill combinations that the current system might miss, further improving job description quality and relevance.

2.6.4 Editing and Version History

Effective job description editing requires maintaining context and allowing users to reference or revert to previous iterations. The version history system addresses this need by systematically storing and managing different states of the job description throughout the editing process.

Purpose and Rationale The primary goal of the version history feature is to provide a robust mechanism for tracking changes, facilitating comparative analysis between different versions, and enabling users to leverage past content during revisions. This supports a non-linear editing workflow where users can experiment with changes, review historical states, and build upon previous iterations without losing work. It directly contributes to the project objective of creating a flexible and user-centric job description editing environment.

Data Structures and Storage Version management is anchored within the ‘Thread’ data model, which encapsulates the entire interaction history for a given session. Key components include:

- **descriptions:** A list within the ‘Thread’ model storing ‘EditedJobDescription’ objects. Each object contains a specific ‘JobDescription’ along with its associated ‘gender_recommendations’.
- **JobDescription:** This Pydantic model includes a crucial ‘version’ field (integer) uniquely identifying each iteration.
- **edit_model_history:** A list within the ‘Thread’ model storing ‘ModelMessage’ objects, capturing the detailed request/response pairs between the user and the editing agent. This provides conversational context.

This structure allows for both discrete snapshots of job descriptions (in ‘descriptions’) and the conversational trail leading to those versions (in ‘edit_model_history’).

```
class JobDescription(BaseModel):
    """Generated job description based on requirements analysis."""
    title: str = Field(description="Job title with seniority level")
    description: str = Field(description="Full job description")
    version: int = Field(description="Version number of the job description")

class EditedJobDescription(BaseModel):
    """Job description with gender recommendations."""
    job_description: JobDescription = Field(description="The job description")
    gender_recommendations: Optional[RecommendationResponse] = Field(
        description="...")

class Thread(BaseModel):
    id: str
    created_at: str
    # ... other fields ...
    edit_model_history: List[ModelMessage] = []
    descriptions: List[EditedJobDescription] = []
```

Agent Integration and Prompting The ‘editor_agent’ is explicitly instructed via its system prompt to utilize the version history stored within the conversational context (‘message_history’).

```
# EDITOR_AGENT_PROMPT snippet
"""
...
3. Version Management:
- all previous job descriptions produced are in the ModelResponse object
  within your message history context
```

```

- Reference specific versions when discussing changes
- If a user asks you to provide an earlier version of the job_description
  use your message history context
- The dynamic prompt will provide you with the current version of the
  job description so use that when filling out the JobDescription version
  field
EXAMPLE:
User: Can you use version 2 of the job description to help me write the
     new one?
Assistant: Retrieve the JobDescription from the message history context
           with version number 2.
- Maintain context across revisions
CRITICAL: Unless specified otherwise by the user assume you are altering
          the
          job description provided by the user in the user prompt.
...
for the version field ensure you return the same version as the provided
job
description, i.e do not alter the version field.
e.g if the user job description has version 1, your output job
description
should also have version 1.
Always return a complete JobDescription object with your changes...
"""

```

This prompt guides the agent to retrieve specific prior versions when requested and to maintain the correct version number when applying edits to an existing version.

API Endpoints and Workflow Several FastAPI endpoints facilitate version management:

- **POST /edit_chat:** Handles general editing requests. It retrieves the relevant 'Thread', passes the 'edit_model_history' to the 'editor_agent', and processes the resulting 'JobDescription'. It uses the version provided in the request's 'current_job_description' for context and maintains this version number in the response unless a new version is explicitly created.
- **POST /create_new_version:** Explicitly creates a new version snapshot. It takes the 'current_job_description', assigns it a new sequential version number (based on the current count in 'thread.descriptions'), and appends the new 'EditedJobDescription' to the 'thread.descriptions' list.
- **POST /save_version:** Updates an *existing* version in the 'thread.descriptions' list. It finds the description matching the provided version number and replaces it with the updated content and recommendations.
- **GET /api/threads/{thread_id}/version-histories:** Retrieves all stored versions ('EditedJobDescription' objects) for a specific thread, allowing the frontend to display the historical progression.

The typical workflow involves:

1. User initiates an edit via `/edit_chat` on the current version.
2. The `editor_agent` uses its `message_history` (containing past interactions and potentially prior `JobDescription` objects) and the `current_job_description` provided in the prompt to perform the edit.
3. The updated `JobDescription` (maintaining the *same* version number) is returned.
4. Optionally, the user saves this as a *new* distinct version via `/create_new_version`, incrementing the version count.
5. The user can view all saved versions via `/api/threads/thread_id/version-histories`.

```
# Inside POST /edit_chat endpoint
async def edit_chat(request: EditChatRequest):
    # ... initialization ...
    thread = Thread(**threads[thread_id])

    # Construct prompt including current version details
    edit_prompt = f"""
    {request.query}

    Current Job Description:
    Title: {request.current_job_description.title}
    {request.current_job_description.description}
    version: {request.current_job_description.version}
    """

    # Pass edit history to the agent
    result = await editor_agent.run(
        edit_prompt,
        deps=deps,
        message_history=thread.edit_model_history # Key for agent context
    )

    updated_job_description = None
    if isinstance(result.data, JobDescription):
        updated_job_description = result.data # Contains the edited
        content

    # ... update thread history, analyze bias ...

    # Return response including the edited job description
    return {
        # ... other fields ...
        "job_description": updated_job_description.model_dump() if
        updated_job_description else None,
```

```

        # ...
    }

# Inside POST /create_new_version endpoint
async def create_new_version(request: CreateNewVersionRequest):
    # ... get thread ...
    thread = Thread(**threads[request.thread_id])

    # Calculate new version number
    new_version = len(thread.descriptions) + 1

    updated_job_description = JobDescription(
        # ... copy details ...
        version=new_version, # Assign new version
    )

    edited_job_description = EditedJobDescription(
        job_description=updated_job_description,
        gender_recommendations=request.gender_recommendations
    )

    # Append to the list of descriptions
    thread.descriptions.append(edited_job_description)

    # ... update storage ...
    return {"job_description": updated_job_description}

```

Limitations and Future Work The current implementation relies on an in-memory dictionary (‘threads’) for storing version history, which is not persistent and limits scalability. Storing full ‘ModelMessage’ history can also become memory-intensive for long conversations. Retrieval complexity for the agent might increase with very long histories.

Future work could involve:

- Implementing persistent storage (e.g., using the existing Supabase integration) for ‘Thread’ data, including the ‘descriptions’ and ‘edit_model_history’.
- Introducing summarisation or pruning strategies for ‘edit_model_history’ to manage context length for the LLM while preserving essential version information.
- Developing a more sophisticated retrieval mechanism for the agent, potentially using vector embeddings of past versions to find relevant historical context more efficiently than linear scanning of message history.

2.7 Rapid Prototyping

Once the agentic back-end was constructed, we needed to develop a front-end interface that ties together the creation of a job description through a chatbot

interface with streamlined editing. As mentioned in section 1.2, one of the key problems with the old MVP product was the lack of in-app editing. Users would have to switch to a different editing app, change their job description, and finally re-upload it for analysis. This fragments their work sphere and is bad design, as the app inhibits uninterrupted creative flow. This fragmentation of the interface has been shown to degrade productivity and user satisfaction across domains. To resolve this, Cycle 2 built upon the agentic system by creating a Streamlit app that unifies generation and editing in one workplace. The following section will discuss the design rationale behind choosing Streamlit for prototype development, followed by a section explaining the implementation.

2.7.1 Design and Rationale

In choosing a rapid prototyping framework for Cycle 2, our goal was to validate the integrated “generate + edit” workflow quickly, gather early stakeholder feedback, and avoid the sunk-cost risks of building a full-scale front end before verifying backend behaviour. Low-fidelity rapid prototyping has been shown to identify major interface flaws swiftly and cheaply, reducing overall development time by up to 50% and avoiding rework costs later in the cycle [58]. To further ensure this prototype could be developed quickly, we decided to go with a Low-code platform which provides pre-built modules and visual components, ensuring all of our efforts could go towards evaluating the performance of the agentic system. The use of such frameworks has been shown to accelerate internal solution delivery [59]. The framework we chose for this prototype was Streamlit, however, there were other alternatives considered:

1. **Dash:** Dash provides a React-based, callback-driven architecture suited to complex, multi-page enterprise dashboards. It offers granular control over layout, theming, and inter-component communication, but at the cost of boilerplate: every UI element and callback must be explicitly defined
2. **Panel:** Panel (part of HoloViz) integrates tightly with the PyData stack, supports both high- and low-level APIs, and can serve Bokeh, Matplotlib, or Plotly charts alongside DataFrame tables. Although its flexibility is compelling, its initial setup and callback wiring introduce significant overhead, making it more suitable for complex analytics apps.
3. **Anvil:** Anvil provides a full drag-and-drop IDE coupled with Python back-end code hosting, enabling non-web developers to build complete apps without JS/HTML. While powerful, it introduces platform lock-in and requires learning Anvil’s proprietary component model.
- 4.

Why Streamlit?

1. **Zero Front-end Context Switch:** Streamlit lets you write both UI and hook into your Python agentic back-end in one file—no separate React/JSX knowledge required

2. **Rapid Iteration with Hot-Reload:** Changes to a Streamlit script reflect instantly in the browser, enabling same-day proof-of-concept demos and fast stakeholder feedback
3. **Built in state management:** Managing *st.session_state* avoids manual workarounds or additional libraries, keeping slot-filling chatbot inputs and incremental edits persistent across interactions
4. **Minimal Setup&Development:** A single pip install streamlit and streamlit run app.py is all that's needed to show stakeholders a working POC, versus multi-package configs or proprietary platforms
5. **Sufficient UI richness:** Streamlit's widgets—text areas, buttons, sliders, and markdown—cover the needs of an inclusive-language editing tool without the overhead of enterprise component libraries

By weighing development speed, ease of integration, and cognitive overhead, Streamlit emerges as the optimal Python framework to rapidly validate and demonstrate your integrated “generate + edit” workflow in Cycle 2.

2.7.2 Implementation of Streamlit Prototype

The implementation follows a stateful, event-driven architecture to manage the conversational flow between the requirements gathering and job description editing phases.

Session State Management The system employs a sophisticated state management approach to maintain context throughout the user interaction journey:

```
if "messages" not in st.session_state:
    st.session_state["messages"] = []

if "edit_messages" not in st.session_state:
    st.session_state.edit_messages = []
    st.session_state.user_messages = []

if "job_descriptions" not in st.session_state:
    st.session_state.job_descriptions = []

if "original_requirements" not in st.session_state:
    st.session_state.original_requirements = None

if "requirements_gathered" not in st.session_state:
    st.session_state.requirements_gathered = False

if "job_description" not in st.session_state:
    st.session_state.job_description = None
```

This state management framework provides four critical capabilities:

1. **Conversational Memory:** The `messages` array maintains a historical record of all interactions, enabling the system to reference previous user inputs when generating contextually appropriate responses. The array stores formatted user and system messages extracted from the function discussed in section 2.6

2. **Version Control:** The `job_descriptions` array maintains a chronological history of all job description iterations, facilitating comparison between versions and allowing users to revert changes if needed.

3. **Phase Tracking:** The `requirements_gathered` boolean flag serves as a finite state machine controller, determining whether the system is in the information collection phase or the refinement phase.

4. **Dual Conversational Contexts:** The system maintains separate message histories for the requirements gathering (`messages`) and editing (`edit_messages`) phases, allowing specialised agent behaviour in each context.

Conversational User Interface

The interface implements a chat-based interaction paradigm, rendering a bidirectional conversation flow through Streamlit's native components:

```
for message in st.session_state["messages"]:  
    if message.user_input:  
        st.chat_message("user").write(message.user_input)  
    if message.system_response:  
        st.chat_message("assistant").write(message.system_response)
```

This presentation approach provides several advantages:

1. **Familiarity:** The chat interface leverages users' existing mental models of conversational agents.

2. **Progressive Disclosure:** Complex job requirements are elicited incrementally through a series of focused questions rather than overwhelming the user with a monolithic form.

3. **Contextual Guidance:** The system can provide immediate feedback and guidance after each user input, creating a collaborative drafting experience.

Bifurcated Interaction Paradigm

The system's interaction flow is bifurcated into two distinct phases with different presentation patterns:

Requirements Gathering Phase

During the initial phase, the system presents a sequential series of questions to elicit job requirements. Input collection occurs through a chat interface that processes natural language responses:

```
if not st.session_state.requirements_gathered:  
    user_input = st.chat_input("Your response:")  
    if user_input:  
        result = await chat(user_input, st.session_state.messages, deps)  
        # Processing logic
```

This phase continues until the system has accumulated sufficient information to generate an initial job description, at which point the `requirements_gathered` flag triggers a transition to the editing phase.

Job Description Editing Phase

Once the initial job description is generated, the interface transforms to present both the current job description and a specialised editing input:

```

if st.session_state.requirements_gathered and st.session_state.
    job_description:
    st.markdown("### Current Job Description")
    st.write(st.session_state.job_description)

    # Display edit message history
    for message in st.session_state.user_messages:
        # Display code

    edit_input = st.chat_input(f"Request changes to the following job
        description:")

```

This phase employs a different conversation model focused on refinement, allowing users to:

1. View the current job description in its entirety
2. Request specific modifications through natural language instructions
3. See a history of their edit requests and the resulting changes
4. Implicitly build a version history as modifications are applied

State Transitions and Data Flow

The transition between phases is determined by the successful collection of structured requirements data:

```

if result.data and isinstance(result.data, JobRequirements):
    st.session_state.requirements_gathered = True
    requirements = result.data

    # Get updated requirements from vector search
    updated_requirements = await analyze_requirements(deps, requirements)

    # Generate job description
    # ...

    st.session_state.job_description = description_result.data.
        description

```

When requirements are successfully structured into a `JobRequirements` object, the system:

1. Updates the state flag to trigger the UI transition
2. Initiates the RAG process through the `analyze_requirements` function
3. Generates the initial job description using both original and RAG-augmented requirements
4. Updates the session state with the generated description

5. Persists the job description to the Supabase backend for future reference

This implementation design creates a seamless, guided experience that progressively transitions from information gathering to collaborative editing, maintaining context throughout the process while presenting an appropriate interface for each phase of the workflow.

2.8 Front End and UI

After concluding the internal test of the system using the prototype streamlit interface, we could commit to building a more fleshed-out interface for user testing. Our findings from internal testing were that the pre-job generation phase, i.e requirements gathering chatbot and the pipeline for producing a job description, were satisfactory. However, the post-editing could do with some improvement from a user experience point of view. The library used for this was React, and we decided to mimic a "canvas" interface for post-job generation editing. The React canvas constitutes Cycle 3 of our Design-Science Research programme and maps to ISO 9241-210 activities "produce design solutions" and "evaluate against requirements". Its purpose is to operationalise the user-centred requirements gathered in Cycles 1–2 by (a) eliminating context-switch costs and (b) exposing agentic reasoning in situ for transparency. The next sub-sections explain how each UI element delivers on those goals.

2.8.1 Design and Rationale

Why React When choosing a front-end framework, we had the following points to consider:

1. **Learning Curve and Self Support:** As I was developing this entirely by myself, with no previous front-end experience, it was crucial for the framework to be relatively easy to learn, and to have a lot of online support
2. **Future Hiring and Maintainability:** JobFair may be looking to expand its team in future, so it was important for the framework to be mainstream, making the hiring process easier.
3. **Ease of Deployment:** Since this is an MVP product, we need to be able to make changes and redeploy the website with ease.

With that in mind, there were several frameworks that were considered but were simply not sufficient in meeting the objectives stated above:

Table 3: Alternative front-end frameworks and their deal-breakers

Framework	Strengths	Deal-breakers for this project
Angular (16+)	Batteries-included; powerful CLI	Steep learning curve (RxJS, DI, Ng-Modules); heavier runtime; smaller talent pool.
Vue 3 / Nuxt	Gentle syntax; good docs	Ecosystem roughly one-third of React; split APIs (Options vs Composition); fewer enterprise widgets.
Svelte / SvelteKit	Near-zero boilerplate; compile-time optimised	Small community; scarce ready-made components (rich-text, analytics); hiring risk.
SolidJS, Qwik, Astro	Ultra-fast hydration; modern DX	Cutting-edge—docs/tooling change monthly; thin ecosystem; little recruiter recognition.
Vanilla JS + Web Components	No framework lock-in; tiny payload	Must hand-roll routing, state-sync, hot-reload, accessibility—too slow for a solo MVP.

With that in mind, React was chosen as it ticks off each one of the requirements stated above:

1. **Learning curve and self-support:** React benefits from the largest ecosystem of free learning resources—over 36 million weekly downloads on npm[60] and 235 000 stars on GitHub[61]—ensuring abundant tutorials, Q&As, and example code.
2. **Future hiring and maintainability:** React is the most in-demand front-end framework, appearing in approximately 52 % of over 250 000 job postings[62], which simplifies recruitment and onboarding.
3. **Ease of deployment:** Next.js (the React meta-framework) provides zero-config, file-system routing out of the box[63] and one-command Vercel deployments that complete in under a minute[64], enabling rapid iterative releases.

Why The Canvas UI After the streamlit prototype, we identified several pain points from the user’s perspective:

1. Users enjoyed having the suggestions displayed, which was present in the old MVP but was removed in the Streamlit prototype
2. We needed explicit version control
3. We needed to have inline editing
4. The agent would sometimes be too liberal with the changes it made

The split canvas UI naturally maps all of these functionalities onto a single coherent page, an alternative was to have several different pages, e.g a version control page, a metrics page and an editor page, however this would still cause fragmentation of the user’s workflow as they would have to switch tasks [31]. By having the chat canvas layout with three strips encapsulating all editing functionality into one page, we turn what would have been an *interruption* into a glancable side interaction, maintaining flow. The split-canvas layout does more than collapse three tools onto one screen; prior research shows that *blending natural-language interaction with persistent, direct-manipulation widgets outperforms a pure chat stream*. *DynaVis* (CHI 2024) let users edit visualisations through both language and auto-synthesised UI controls; in a 24-person study, participants *preferred DynaVis to an NLI-only interface* because the on-canvas widgets made subsequent tweaks easier and increased confidence via immediate visual feedback [65]. Likewise, the *Collaborative Canvas* probe (IUI Workshops 2024) showed that teams valued AI-generated sticky notes but wanted a shared canvas so they could *review and filter* suggestions before exposing them to the group [66]. The success and findings of these projects support our canvas design as a more preferable way of collaborating with LLM. The remainder of this design section will justify design choices for each of the strips shown in *add figure*

Strip-1 -Chat to Edit Pane This is the main pane through which the user can utilise the LLM for editing. The rationale behind including an LLM for editing is simple; these models are effective at editing structured and semi-structured documents using only simple prompts [67]. Although the Weber study found the LLM’s performance on editing tasks to be linked with how well the document was structured, this wasn’t a concern, as the job description produced from the agentic pipeline already produced a well-structured document with headings. Utilising LLM for document editing in particular has been shown to boost productivity by reducing manual effort from the users’ side [68]. This design choice contributes to solving the previously identified problem of the MVP lacking seamless in-context editor functionality.

A noticeable change we made to this AI editor agent was changing the system prompt from the Streamlit Prototype. Previously the agent would take in a user query and automatically make changes using gender neutral language. However after internal testing the CTO of the company stated he preferred allowing users to make the exact changes they wanted, and to be given the options to make these changes gender neutral. This gives the user even more fine grained control over which changes they want to apply.

Strip-2 Canvas Editing This strip is where the currently selected job description is displayed. The canvas supports an edit mode which allows the user to freely edit the job description. The rationale behind this functionality was based on the Collaborative Canvas study [66], which concluded that although users enjoyed the assistance of LLMS, they wanted to have the freedom to manually

review and make changes to the generated output. While in editing mode, the user can select text for "AI editing"; this mode allows for selecting specific text which the edit agent should change. This provides a more fine-grained method of editing in addition to using the LLM agent in the chat window, which will try to make changes. However, as noted in the reflection section of the streamlit prototype, we found that the agent was sometimes too liberal with changing text. A pure chat prompt asks the LLM to "guess" the edit span; it therefore tends to rewrite more text than the user intended—a problem we observed in the Streamlit pilot during internal testing. Highlight-to-edit removes that ambiguity by binding the selection range to the agent request, which delivers two advantages backed by recent studies:

1. InkSync’s two controlled studies showed that interfaces which bind executable edits to a user’s text selection lead to significantly faster revisions and fewer factual errors than chat-only editing [69]
2. In the collaborative canvas user reported that they trusted the AI suggestions only after they could "pinpoint exactly where the changes would go" and keep the rest of the document intact [66] Our select-to-edit mode mirrors that affordance, letting recruiters keep control of tone while still leveraging the agent for micro-edits.

By embedding this highlight workflow directly in the canvas, we give users a *precision scalpel* instead of a *broad brush*, correcting the over-editing pain surfaced in the prototype and aligning with evidence that selection-scoped commands improve both accuracy and user confidence.

The last feature we implemented was changing the way versions were being managed compared to the Streamlit Prototype. In the Streamlit prototype, every agent edit spawned a new revision, quickly flooding the history and obscuring which snapshots actually mattered. In the React canvas, we switched to an explicit "save → branch" model, like Git: edits remain ephemeral until the user clicks Save, and a Create Version button checkpoints the current state under a new version ID. This puts authorship back in the recruiter’s hands and yields two documented benefits:

1. **Meaningful landmarks:** Creative-work studies show that practitioners prefer to checkpoint at self-defined "conceptual milestones", not after every micro-change; granular logs degrade recall and increase retrieval time [44]
2. **Safer exploration:** When versions are user-initiated, recruiters can run bold AI edits, review the result, and abandon it without polluting the timeline—an interaction pattern that Chan & Schunn found to encourage risk-taking and ultimately higher-quality outcomes in iterative tasks [45]

Strip-3 Version—Metrics Strip The right-hand strip couples two elements—version timeline and live-metrics panel—to support reflection-in-action without forcing a context switch.

Version timeline. Each commit appears as a collapsible card labelled “v1”, “v2”, Expanding a card reveals the inclusive-language suggestions generated by the agentic judge together with a short rationale (e.g. “Avoids masculine pronoun”). Clicking a suggestion highlights the affected span on the canvas, allowing the recruiter to inspect or accept it in place. Providing the explanatory snippet preserves JobFair’s pedagogy goal: concise “why/how” explanations have been shown to raise user trust and perceived usefulness compared with black-box corrections[3]. Because commits are created only when the user presses *Create Version*, the timeline contains meaningful milestones rather than a flood of auto-saves as mentioned in section 2.8.1

Live-metrics panel. Beneath the timeline, a small dashboard streams the current Flesch Reading Ease and word count as the recruiter types. Instant, surface-level feedback nudges authors toward clearer, shorter prose: texts above FRE 60 are consistently rated easier to comprehend by broad audiences[?]. Displaying the scores in real time turns readability from a late validation step into a continuous guidance cue, letting recruiters converge on an accessible, inclusive draft without leaving the canvas.

References

- [1] D. A. Collier and C. Zhang, “Can we reduce bias in the recruiting process and diversify pools of candidates by using different types of words in job descriptions,” 2016.
- [2] T. Romano, F. Mohammadi, and P. Ceravolo, “Synthetic data for identifying inclusive language (case study: Job descriptions in italian),” *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, pp. 737–742, 2024.
- [3] A. Kartikeya, “Examining correlation between trust and transparency with explainable artificial intelligence,” 2021.
- [4] M. Guesmi, M. A. Chatti, S. Joarder, Q. U. Ain, C. Siepmann, H. Ghanbarzadeh, and R. Alatrash, “Justification vs. transparency: Why and how visual explanations in a scientific literature recommender system,” 2023.
- [5] G. Mark, V. M. Gonzalez, and J. Harris, “No task left behind? examining the nature of fragmented work,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI ’05, (New York, NY, USA), p. 321–330, Association for Computing Machinery, 2005.
- [6] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

- [7] “ISO 9241-210:2019 ergonomics of human-system interaction – part210: Human-centred design for interactive systems,” 2019. Accessed 17 April 2025.
- [8] K. Chang, S. Xu, C. Wang, Y. Luo, T. Xiao, and J. Zhu, “Efficient prompting methods for large language models: A survey,” *ArXiv*, vol. abs/2404.01077, 2024.
- [9] J. Zhuo, S. Zhang, X. Fang, H. Duan, D. Lin, and K. Chen, “Prosa: Assessing and understanding the prompt sensitivity of llms,” *ArXiv*, vol. abs/2410.12405, 2024.
- [10] M. Rana, K. Hacioglu, S. Gopalan, and M. Boothalingam, “Zero-shot slot filling in the age of llms for dialogue systems,” *ArXiv*, vol. abs/2411.18980, 2024.
- [11] T. Rietz and A. Maedche, “Ladderbot: A requirements self-elicitation system,” 09 2019.
- [12] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, “A systematic survey of prompt engineering in large language models: Techniques and applications,” 2025.
- [13] C. Sadasivan, C. Cruz, N. Dolgoy, A. Hyde, S. Campbell, M. McNeely, E. Stroulia, and P. Tandon, “Examining patient engagement in chatbot development approaches for healthy lifestyle and mental wellness interventions: Scoping review,” *J Particip Med*, vol. 15, p. e45772, May 2023.
- [14] A. Razavi, M. Soltangheis, N. Arabzadeh, S. Salamat, M. Zihayat, and E. Bagheri, “Benchmarking prompt sensitivity in large language models,” 2025.
- [15] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting,” 2024.
- [16] C. Bonial and H. T. Madabushi, “A construction grammar corpus of varying schematicity: A dataset for the evaluation of abstractions in language models,” pp. 243–255, 2024.
- [17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021.
- [18] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval augmentation reduces hallucination in conversation,” 2021.
- [19] D. J. Shah, R. Gupta, A. Fayazi, and D. Z. Hakkani-Tür, “Robust zero-shot cross-domain slot filling with example values,” *ArXiv*, vol. abs/1906.06870, 2019.

- [20] R. Pimplikar, K. Kannan, A. Mondal, J. Mondal, S. Saxena, G. Parija, and C. Devulapalli, “Rise: Resolution of identity through similarity establishment on unstructured job descriptions,” pp. 19–36, 2017.
- [21] N. Dixon, M. Goggins, E. Ho, M. Howison, J. Long, E. Northcott, K. Shen, and C. Yeats, “Occupational models from 42 million unstructured job postings,” *Patterns*, vol. 4, no. 7, p. 100757, 2023.
- [22] E. R. Adhikari, “Understanding nuances and commonalities of job descriptions,” vol. 1, pp. 67–76, 2015.
- [23] R. A. Cheramie, “Improving career development in students by developing job analysis skills (forthcoming),” 2014.
- [24] K. Pradeep, “Personal preference in choosing a job,” *International Journal of Innovative Technology and Exploring Engineering*, 2019.
- [25] M. Gan and B. H. Kleiner, “How to write job descriptions effectively,” *Management Research News*, 2005.
- [26] T. Abdelaziz, A. M. Maatuk, and F. Rajab, “An approach to improvement the usability in software products,” *International Journal of Software Engineering Applications*, vol. 7, pp. 11–18, 2016.
- [27] H. Soni, J. Ivanova, H. Wilczewski, A. Bailey, T. Ong, A. Narma, B. E. Bunnell, and B. M. Welch, “Virtual conversational agents versus online forms: Patient experience and preferences for health data collection,” *Frontiers in digital health*, vol. 4, 2022.
- [28] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, “Active retrieval augmented generation,” *ArXiv*, vol. abs/2305.06983, 2023.
- [29] S. Sivarajkumar, M. Kelley, A. Samolyk-Mazzanti, S. Visweswaran, and Y. Wang, “An empirical evaluation of prompting strategies for large language models in zero-shot clinical natural language processing: Algorithm development and validation study,” *JMIR Medical Informatics*, vol. 12, 2024.
- [30] Y. Li, R. Ramprasad, and C. Zhang, “A simple but effective approach to improve structured language model output for information extraction,” pp. 5133–5148, 2024.
- [31] G. Mark, V. M. González, and J. Harris, “No task left behind?: examining the nature of fragmented work,” *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2005.
- [32] Z. Gou, Z. Shao, Y. Gong, yelong shen, Y. Yang, N. Duan, and W. Chen, “CRITIC: Large language models can self-correct with tool-interactive critiquing,” in *The Twelfth International Conference on Learning Representations*, 2024.

- [33] A. Khan, J. Hughes, D. Valentine, L. Ruis, K. Sachan, A. Radhakrishnan, E. Grefenstette, S. R. Bowman, T. Rocktäschel, and E. Perez, “Debating with more persuasive llms leads to more truthful answers,” 2024.
- [34] L. N. Lior, “Chapter 6 - understanding user reading patterns,” in *Writing for Interaction* (L. N. Lior, ed.), pp. 91–106, Boston: Morgan Kaufmann, 2013.
- [35] Government Digital Service, “Content design: planning, writing and managing content.” <https://www.gov.uk/guidance/content-design>, 2024. Accessed 26 Apr 2025.
- [36] World Wide Web Consortium (W3C), “Web content accessibility guidelines (wcag) 2.2,” Oct. 2023. W3C Recommendation.
- [37] D. Eleyan, A. Othman, and A. Eleyan, “Enhancing software comments readability using flesch reading ease score,” *Inf.*, vol. 11, p. 430, 2020.
- [38] H. Leung, “The impact of information quality of job descriptions on an applicant’s decision to pursue a job,” 2007.
- [39] U. Lagowska, F. Sobral, J. Jacob, A. Hafenbrack, and R. Goldszmidt, “Following community norms or an internal compass? the role of prospective leaders’ social category membership in the differential effects of authentic and ethical leadership on stereotype threat,” *The Journal of applied psychology*, 2023.
- [40] R. K. Sawyer, “The iterative and improvisational nature of the creative process,” *Journal of Creativity*, vol. 31, p. 100002, 2021.
- [41] J. Schmidhuber, S. Schlogl, and C. Ploder, “Cognitive load and productivity implications in human-chatbot interaction,” in *2021 IEEE 2nd International Conference on Human-Machine Systems (ICHMS)*, p. 1–6, IEEE, Sept. 2021.
- [42] R. Tanaka, A. Ozeki, S. Kato, and A. Lee, “Context and knowledge aware conversational model and system combination for grounded response generation,” *Computer Speech Language*, vol. 62, p. 101070, 2020.
- [43] M. Kleppmann, W. Maughan, and D. Grover, “Upwelling: Combining real-time collaboration with version control for writers.” Ink & Switch Research Lab Technical Report, 2023. Accessed 25 Apr 2025.
- [44] S. Sterman, M. J. Nicholas, and E. Paulos, “Towards creative version control,” *Proc. ACM Hum.-Comput. Interact.*, vol. 6, Nov. 2022.
- [45] J. Chan and C. D. Schunn, “The importance of iteration in creative conceptual combination,” *Cognition*, vol. 145, pp. 104–115, 2015.

- [46] M. Varsha, S. Yashashree, D. K. Ramdas, and S. Alex, “A review of existing approaches to increase the computational speed of the python language,” 2019.
- [47] F. Zehra, M. Javed, D. Khan, and M. Pasha, “Comparative analysis of c++ and python in terms of memory and time,” 2020.
- [48] Y. Zhang, “Comparative study of the execution efficiency of python and c++based on topological sorting,” *Applied and Computational Engineering*, 2024.
- [49] Stack Overflow, “Stack overflow developer survey 2024.” <https://insights.stackoverflow.com/survey/2024>, 2024. Accessed: 17 April 2025.
- [50] J.-S. Lee, T. Filatova, A. Ligmann-Zielinska, B. Hassani-Mahmooei, F. Stonedahl, I. Lorscheid, A. Voinov, J. G. Polhill, Z. Sun, and D. Parker, “The complexities of agent-based modeling output analysis,” *J. Artif. Soc. Soc. Simul.*, vol. 18, 2015.
- [51] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, “Prompt injection attack against llm-integrated applications,” *ArXiv*, vol. abs/2306.05499, 2023.
- [52] N. Ligeti-Nagy and Z. G. Yang, “Improve performance of fine-tuning language models with prompting,” *Infocommunications journal*, 2023.
- [53] Pydantic AI Team, “Accessing messages from results.” <https://ai.pydantic.dev/message-history/#accessing-messages-from-results>, 2025. Accessed: 2025-04-24.
- [54] K. T. iunn Ong, T. Kwon, and J. Yeo, “Large language models are self-taught reasoners: Enhancing llm applications via tailored problem-solving demonstrations,” *ArXiv*, vol. abs/2408.12315, 2024.
- [55] A. Drozdov, H. Zhuang, Z. Dai, Z. Qin, R. Rahimi, X. Wang, D. Alon, M. Iyyer, A. McCallum, D. Metzler, and K. Hui, “Parade: Passage ranking using demonstrations with large language models,” *ArXiv*, vol. abs/2310.14408, 2023.
- [56] H. Gao, F. Zhang, W. Jiang, J. Shu, F. Zheng, and H. Wei, “On the noise robustness of in-context learning for text generation,” *ArXiv*, vol. abs/2405.17264, 2024.
- [57] L. Wang, N. Yang, and F. Wei, “Learning to retrieve in-context examples for large language models,” pp. 1752–1767, 2023.
- [58] S. D. Nelson, G. Del Fioli, H. Hanseler, B. I. Crouch, and M. R. Cummins, “Software prototyping: A case report of refining user requirements for a health information exchange dashboard,” *Applied Clinical Informatics*, vol. 7, no. 1, pp. 22–32, 2016.

- [59] N. Upadhyaya, “Low-code/no-code platforms and their impact on traditional software development: A literature review,” 03 2023.
- [60] “React package on npm.” <https://www.npmjs.com/package/react>, 2025. Weekly downloads: 36,131,317.
- [61] Facebook, Inc., “facebook/react: The library for web and native user interfaces.” <https://github.com/facebook/react>, 2025. Stars: 235k.
- [62] Logan, “The most demanded frontend frameworks in 2024.” <https://www.devjobsscanner.com/blog/the-most-demanded-frontend-frameworks/>, 2025. React accounts for 52% of 250k frontend job offers.
- [63] “Routing — next.js documentation.” <https://nextjs.org/docs/pages/building-your-application/routing>, 2025.
- [64] “Deploying next.js to vercel — next.js learn.” <https://nextjs.org/learn/pages-router/deploying-nextjs-app-deploy>, 2025. Deployment finishes in under a minute.
- [65] P. Vaithilingam, E. L. Glassman, J. Inala, and C. Wang, “Dynavis: Dynamically synthesized ui widgets for visualization editing,” *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024.
- [66] G. Gonzalez, D. A. S. Moran, S. Houde, J. He, S. I. Ross, M. J. Muller, S. Kunde, and J. D. Weisz, “Collaborative canvas: A tool for exploring llm use in group ideation tasks,” 2024.
- [67] I. Weber, “Large language models are pattern matchers: Editing semi-structured and structured documents with chatgpt,” pp. 3–18, 2024.
- [68] S. Koneru, M. Exel, M. Huck, and J. Niehues, “Contextual refinement of translations: Large language models for sentence and document-level post-editing,” *ArXiv*, vol. abs/2310.14855, 2023.
- [69] P. Laban, J. Vig, M. A. Hearst, C. Xiong, and C.-S. Wu, “Beyond the chat: Executable and verifiable text-editing with LLMs,” in *UIST ’24: 37th Annual ACM Symposium on User Interface Software and Technology*, 2024.